Programming Fundamentals

Week 6 — Building a CRUD Application

Programming Fundamentals Team

SETU

2026-10-20





Last Time

We combined concepts we already knew:

lists + dictionaries + loops + selection + functions



Last Time

We combined concepts we already knew:

lists + dictionaries + loops + selection + functions and introduced:

Create — Read — Update — Delete

Our Week 6 Project: Dog Day Care Manager





Imagine you work at a **Dog Day Care Centre**. Staff need a simple program to keep track of the dogs attending.

Each dog has an **ID, name, breed and age**.

Our program will let us **add, view, find, update and delete** dog records.

Our Week 6 Project: Dog Day Care Manager

The programming goal: practise CRUD using **lists of dictionaries, functions and a menu**.

Our Dog Day Care Example

We will build a small **Dog Day Care Manager**.

```
dogs = [
    {"id": 1, "name": "Buddy", "breed": "Golden Retriever", "age": 4},
    {"id": 2, "name": "Luna", "breed": "Labrador", "age": 2}
]
```

Our Dog Day Care Example

We will build a small **Dog Day Care Manager**.

```
dogs = [
    {"id": 1, "name": "Buddy", "breed": "Golden Retriever", "age": 4},
    {"id": 2, "name": "Luna", "breed": "Labrador", "age": 2}
]
```

Today the data exists only while the program is running.



Start with Read

Before changing data, make sure we can display it clearly.

```
def display_dogs(dogs):
    for dog in dogs:
        print(dog["id"], dog["name"], dog["breed"], dog["age"])
```



Start with Read

Before changing data, make sure we can display it clearly.

```
def display_dogs(dogs):
    for dog in dogs:
        print(dog["id"], dog["name"], dog["breed"], dog["age"])
```

Why pass `dogs` as a parameter?

Find One Record

Several CRUD operations need to find a dog first.

```
def find_dog(dogs, dog_id):
    for dog in dogs:
        if dog["id"] == dog_id:
            return dog

    return None
```

Find One Record

Several CRUD operations need to find a dog first.

```
def find_dog(dogs, dog_id):
    for dog in dogs:
        if dog["id"] == dog_id:
            return dog

    return None
```

Write the search once. Reuse it wherever it is needed.

Using None

```
dog = find_dog(dogs, 2)

if dog is not None:
    print(dog["name"])
else:
    print("Dog not found")
```



Using None

```
dog = find_dog(dogs, 2)

if dog is not None:
    print(dog["name"])
else:
    print("Dog not found")
```

`None` means that the function did not find a matching record.



Create — Add a New Record

```
def add_dog(dogs):
    dog_id = int(input("ID: "))
    name = input("Name: ")
    breed = input("Breed: ")
    age = int(input("Age: "))

    dog = {
        "id": dog_id,
        "name": name,
        "breed": breed,
        "age": age
    }

    dogs.append(dog)
```

Create — What Could Go Wrong?

Before adding a dog, ask:

Should two dogs be allowed to have the same ID?



Create — What Could Go Wrong?

Before adding a dog, ask:

Should two dogs be allowed to have the same ID?

```
existing = find_dog(dogs, dog_id)

if existing is not None:
    print("That ID already exists")
else:
    # create and append the new dog
```

Create — What Could Go Wrong?

Before adding a dog, ask:

Should two dogs be allowed to have the same ID?

```
existing = find_dog(dogs, dog_id)

if existing is not None:
    print("That ID already exists")
else:
    # create and append the new dog
```

We are combining **Create** with **Read/search**.

Update — Find, Then Change

```
def update_dog(dogs):
    dog_id = int(input("Dog ID: "))
    dog = find_dog(dogs, dog_id)

    if dog is not None:
        dog["age"] = int(input("New age: "))
        print("Dog updated")
    else:
        print("Dog not found")
```

Update — Find, Then Change

```
def update_dog(dogs):
    dog_id = int(input("Dog ID: "))
    dog = find_dog(dogs, dog_id)

    if dog is not None:
        dog["age"] = int(input("New age: "))
        print("Dog updated")
    else:
        print("Dog not found")
```

We don't create a new dictionary — we change the existing one.



Update More Than One Field?

We could also allow the user to update other values:

```
dog["name"] = input("New name: ")
dog["breed"] = input("New breed: ")
dog["age"] = int(input("New age: "))
```



Update More Than One Field?

We could also allow the user to update other values:

```
dog["name"] = input("New name: ")
dog["breed"] = input("New breed: ")
dog["age"] = int(input("New age: "))
```

Design question: should the user have to change **everything**?

There is often more than one reasonable solution.

Delete — Find, Then Remove

```
def delete_dog(dogs):
    dog_id = int(input("Dog ID: "))
    dog = find_dog(dogs, dog_id)

    if dog is not None:
        dogs.remove(dog)
        print("Dog deleted")
    else:
        print("Dog not found")
```

CRUD Functions

Our program now has clear operations:

```
add_dog()       → Create
display_dogs()  → Read
find_dog()      → Read/Search
update_dog()    → Update
delete_dog()    → Delete
```



CRUD Functions

Our program now has clear operations:

```
add_dog()       → Create
display_dogs()  → Read
find_dog()      → Read/Search
update_dog()    → Update
delete_dog()    → Delete
```

Each function has a clear responsibility.



How Does the User Choose?

We already know how to build this:

```
print("1. Add dog")
print("2. View dogs")
print("3. Update dog")
print("4. Delete dog")
print("0. Exit")

choice = input("Choose an option: ")
```



How Does the User Choose?

We already know how to build this:

```
print("1. Add dog")
print("2. View dogs")
print("3. Update dog")
print("4. Delete dog")
print("0. Exit")

choice = input("Choose an option: ")
```

What do we need if we want the menu to appear again?

A Menu Loop

```
choice = ""

while choice != "0":
    print("1. Add dog")
    print("2. View dogs")
    print("3. Update dog")
    print("4. Delete dog")
    print("0. Exit")

    choice = input("Choose an option: ")
```

A Menu Loop

```
choice = ""

while choice != "0":
    print("1. Add dog")
    print("2. View dogs")
    print("3. Update dog")
    print("4. Delete dog")
    print("0. Exit")

    choice = input("Choose an option: ")
```

This is why we learned `while` loops.

Connect the Menu to the Functions

```
if choice == "1":
    add_dog(dogs)
elif choice == "2":
    display_dogs(dogs)
elif choice == "3":
    update_dog(dogs)
elif choice == "4":
    delete_dog(dogs)
elif choice != "0":
    print("Invalid option")
```

Connect the Menu to the Functions

```
if choice == "1":
    add_dog(dogs)
elif choice == "2":
    display_dogs(dogs)
elif choice == "3":
    update_dog(dogs)
elif choice == "4":
    delete_dog(dogs)
elif choice != "0":
    print("Invalid option")
```

Selection decides which operation to perform.

Look at What We Have Combined

Our small CRUD application uses:

Concept	Job in the program
Variables	Store individual values
Selection	Choose actions and handle decisions
Functions	Organise operations
Iteration	Repeat menus and process records
Lists	Store the collection
Dictionaries	Represent each record



Important Limitation

Add a dog, then stop the program and run it again.

What happens?



Important Limitation

Add a dog, then stop the program and run it again.

What happens? **The new dog is gone.**

Important Limitation

Add a dog, then stop the program and run it again.

What happens? **The new dog is gone.** Our list only exists in memory while Python is running.

This gives us our next problem to solve: persistence using files / JSON.

Mini Activity — Trace a CRUD Operation

Choose one operation: **Create, Read, Update or Delete**.

Write its steps in plain English before looking at the Python.

Example for Update:

```
1. Ask for the ID
2. Search for the record
3. If found, ask for the new value
4. Change the record
5. Otherwise display an error
```

Mini Activity — Trace a CRUD Operation

Choose one operation: **Create, Read, Update or Delete**.

Write its steps in plain English before looking at the Python.

Example for Update:

```
1. Ask for the ID
2. Search for the record
3. If found, ask for the new value
4. Change the record
5. Otherwise display an error
```

Then identify where selection, iteration and functions are needed.

Week 6 — Key Takeaways

- CRUD is a pattern for managing collections of data
- CRUD is not a new Python feature
- It combines concepts we already know
- Searching is an important reusable operation
- Functions keep each operation manageable
- Our data currently disappears when the program ends

Week 6 — Key Takeaways

- CRUD is a pattern for managing collections of data
- CRUD is not a new Python feature
- It combines concepts we already know
- Searching is an important reusable operation
- Functions keep each operation manageable
- Our data currently disappears when the program ends

We now have the foundations for a much more realistic data-driven program.